

ASSIGNMENT-13.5

Name : Sahasra

Roll no : 2303A51269

B19

Lab 13: Code Refactoring – Improving Legacy Code with AI

Suggestions

Lab Objectives:

Week7 -

Friday

- Identify code smells and inefficiencies in legacy Python scripts.
- Use AI-assisted coding tools to refactor for readability, maintainability, and performance.
- Apply modern Python best practices while ensuring output correctness.

Task Description #1 (Refactoring – Removing Global Variables)

- Task: Use AI to eliminate unnecessary global variables from the code.
- Instructions:
 - o Identify global variables used across functions.
 - o Refactor the code to pass values using function parameters.
 - o Improve modularity and testability.
- Sample Legacy Code:
rate = 0.1

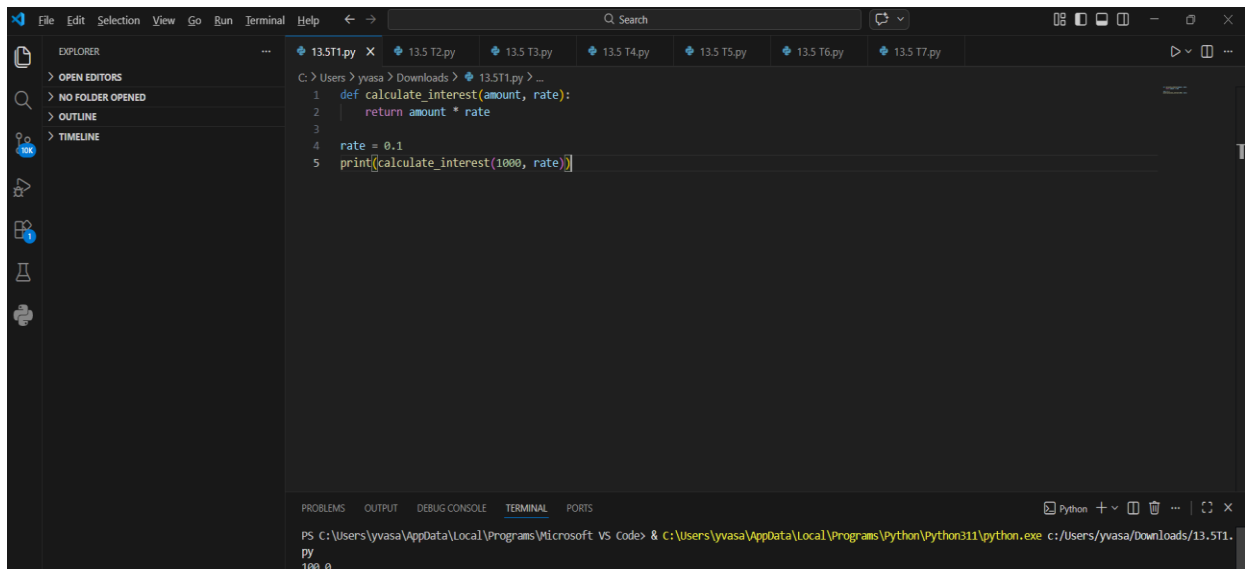
def calculate_interest(amount):

```
return amount * rate

print(calculate_interest(1000))
```

- Expected Output:

o Refactored version passing rate as a parameter or using a configuration structure.



Task Description #2 : (Refactoring Deeply Nested Conditionals)

- Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.

- Focus Areas:

o Readability

o Logical simplification

o Maintainability

Legacy Code:

```
score = 78
```

```
if score >= 90:
```

```
    print("Excellent")
```

```
else:
```

```
    if score >= 75:
```

```
        print("Very Good")
```

else:

if score >= 60:

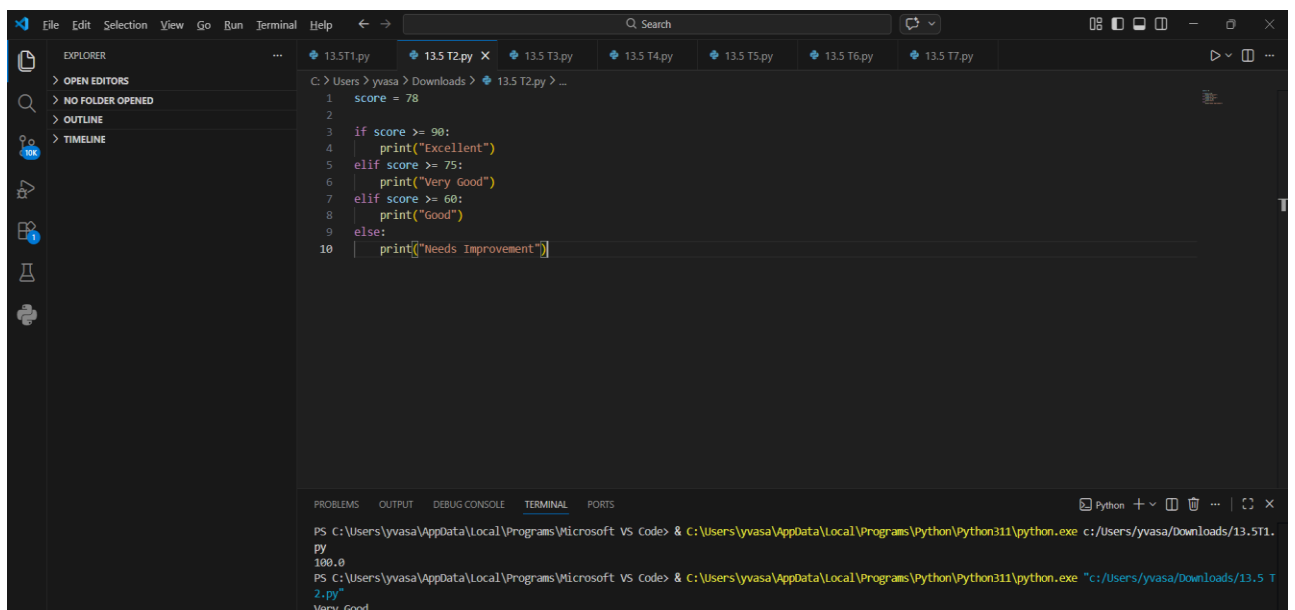
print("Good")

else:

print("Needs Improvement")

Expected Outcome:

o Flattened logic using guard clauses or a mapping-based approach.



The screenshot shows a Visual Studio Code editor window with a Python file named 13.5 T2.py. The code in the editor is as follows:

```
1 score = 78
2
3 if score >= 90:
4     print("Excellent")
5 elif score >= 75:
6     print("Very Good")
7 elif score >= 60:
8     print("Good")
9 else:
10    print("Needs Improvement")
```

The terminal at the bottom shows the command to run the script and its output:

```
PS C:\Users\yvasa\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yvasa\AppData\Local\Programs\Python\Python311\python.exe c:/Users/yvasa/Downloads/13.5 T2.py
Very Good
```

Task 3 (Refactoring Repeated File Handling Code)

- Task: Use AI to refactor repeated file open/read/close logic.

- Focus Areas:

- o DRY principle

- o Context managers

- o Function reuse

Legacy Code:

```
f = open("data1.txt")
```

```
print(f.read())
```

```
f.close()
```

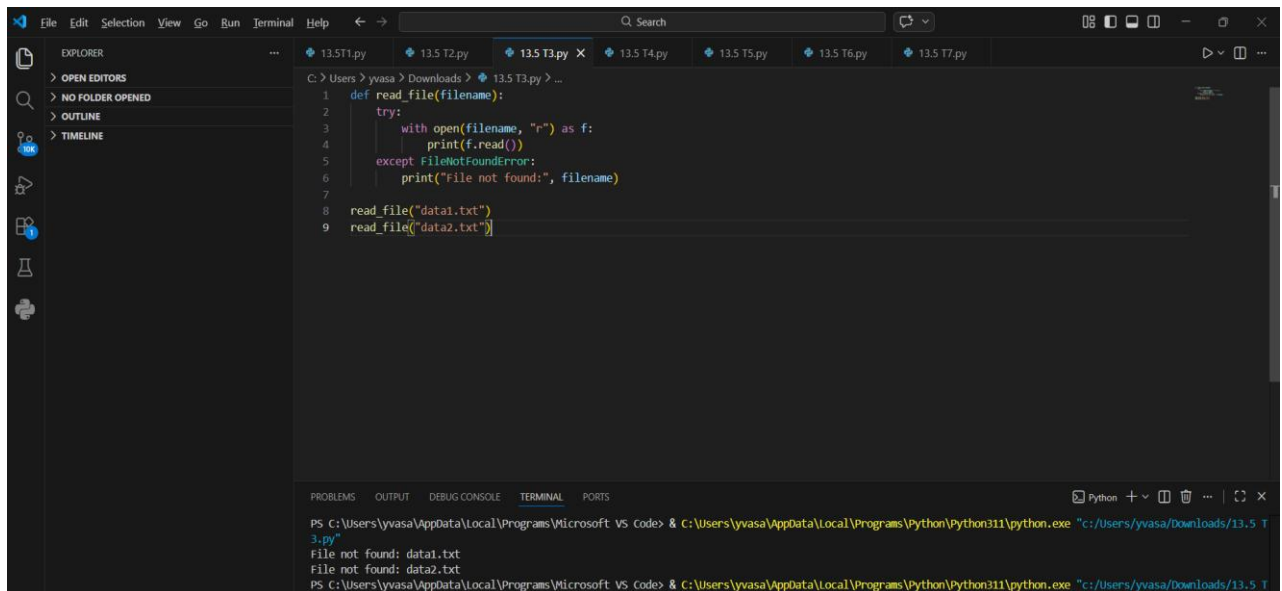
```
f = open("data2.txt")
```

```
print(f.read())
```

```
f.close()
```

Expected Outcome:

o Reusable function using with open() and parameters.



The screenshot shows a Visual Studio Code editor window with a Python file named 13.5 T3.py. The code defines a function read_file(filename) that uses a try-except block to attempt opening a file with 'r' mode. If successful, it prints the file's content; if a FileNotFoundError occurs, it prints a message. The function is then called twice with 'data1.txt' and 'data2.txt'. The terminal at the bottom shows the command prompt running the script, which outputs 'File not found: data1.txt' and 'File not found: data2.txt'.

```
1 def read_file(filename):
2     try:
3         with open(filename, "r") as f:
4             print(f.read())
5     except FileNotFoundError:
6         print("File not found:", filename)
7
8 read_file("data1.txt")
9 read_file("data2.txt")
```

```
PS C:\Users\yvasa\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yvasa\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/yvasa/Downloads/13.5 T3.py"
File not found: data1.txt
File not found: data2.txt
PS C:\Users\yvasa\AppData\Local\Programs\Microsoft VS Code>
```

Task 4 (Optimizing Search Logic)

- Task: Refactor inefficient linear searches using appropriate

data structures.

- Focus Areas:

- o Time complexity

- o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
```

```
name = input("Enter username: ")
```

```
found = False
```

```
for u in users:
```

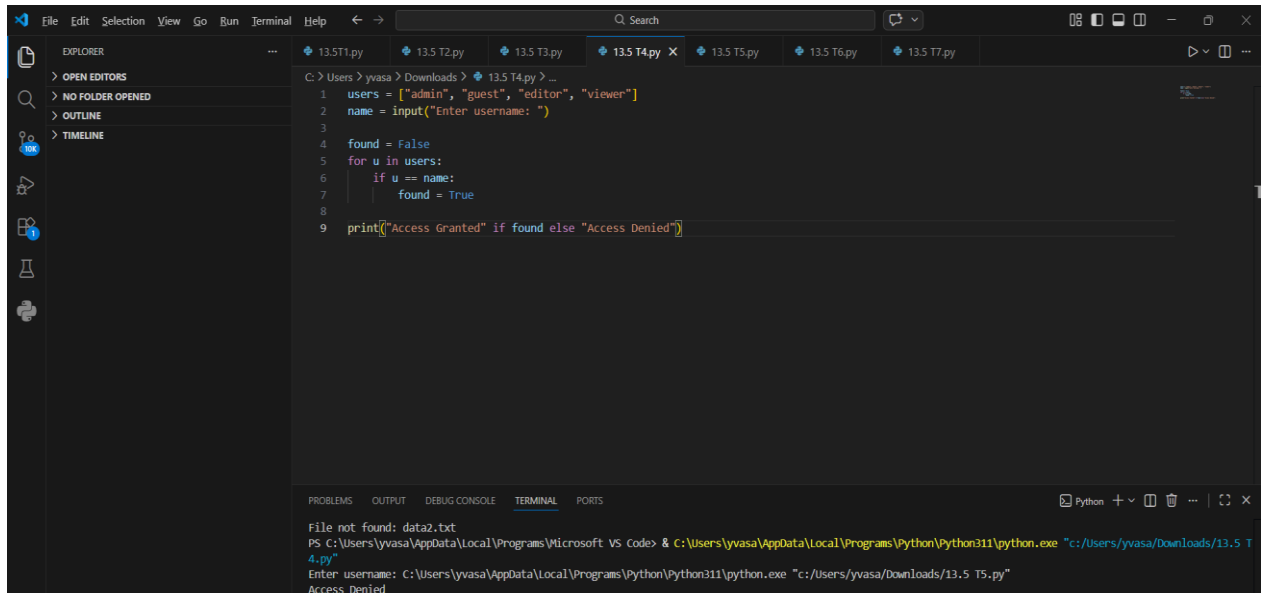
```
    if u == name:
```

```
        found = True
```

```
print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

- o Use of sets or dictionaries with complexity justification.



The screenshot shows a VS Code editor with a Python file named 13.5 T4.py. The code defines a list of users and a function to check if a username is in the list. The terminal output shows the execution of the script, which prompts for a username and outputs "Access Denied".

```
C:\Users\yvasa> Downloads > 13.5 T4.py > ...
1  users = ["admin", "guest", "editor", "viewer"]
2  name = input("Enter username: ")
3
4  found = False
5  for u in users:
6      if u == name:
7          found = True
8
9  print("Access Granted" if found else "Access Denied")
```

File not found: data2.txt
PS C:\Users\yvasa\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yvasa\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/yvasa/Downloads/13.5 T4.py"
Enter username: c:\Users\yvasa\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/yvasa/Downloads/13.5 T5.py"
Access Denied

Task 5 (Refactoring Procedural Code into OOP Design)

- Task: Use AI to refactor procedural code into a class-based

design.

- Focus Areas:

- o Object-Oriented principles

- o Encapsulation

Legacy Code:

```
salary = 50000
```

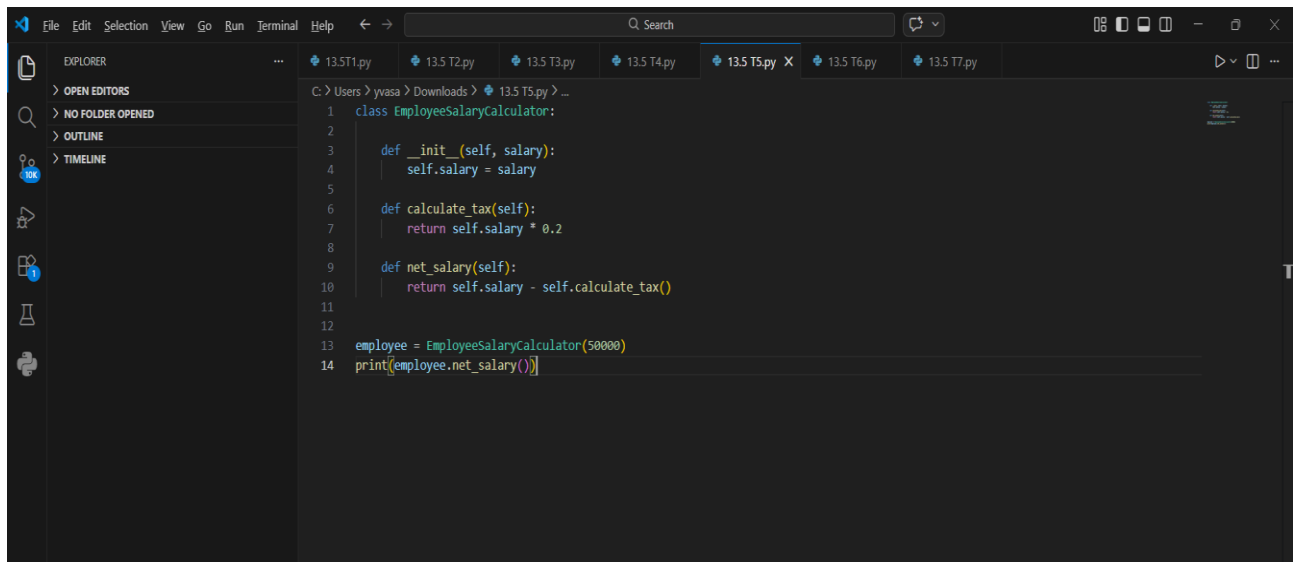
```
tax = salary * 0.2
```

```
net = salary - tax
```

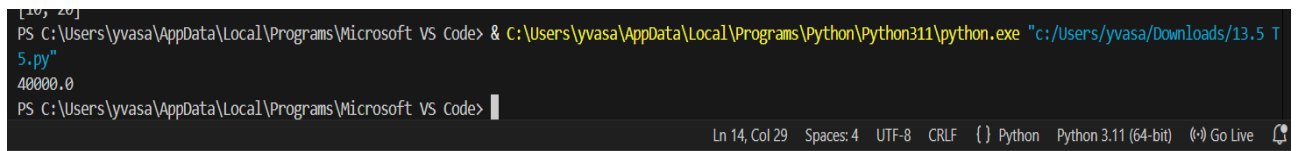
```
print(net)
```

Expected Outcome:

- o A class like EmployeeSalaryCalculator with methods and attributes.



```
1 class EmployeeSalaryCalculator:
2
3     def __init__(self, salary):
4         self.salary = salary
5
6     def calculate_tax(self):
7         return self.salary * 0.2
8
9     def net_salary(self):
10        return self.salary - self.calculate_tax()
11
12
13 employee = EmployeeSalaryCalculator(50000)
14 print(employee.net_salary())
```



```
PS C:\Users\yvasa\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yvasa\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/yvasa/Downloads/13.5 T5.py"
40000.0
PS C:\Users\yvasa\AppData\Local\Programs\Microsoft VS Code>
```

Task 6 (Refactoring for Performance Optimization)

- Task: Use AI to refactor a performance-heavy loop handling large data.

- Focus Areas:

- o Algorithmic optimization

- o Use of built-in functions

Legacy Code:

```
total = 0
```

```
for i in range(1, 1000000):
```

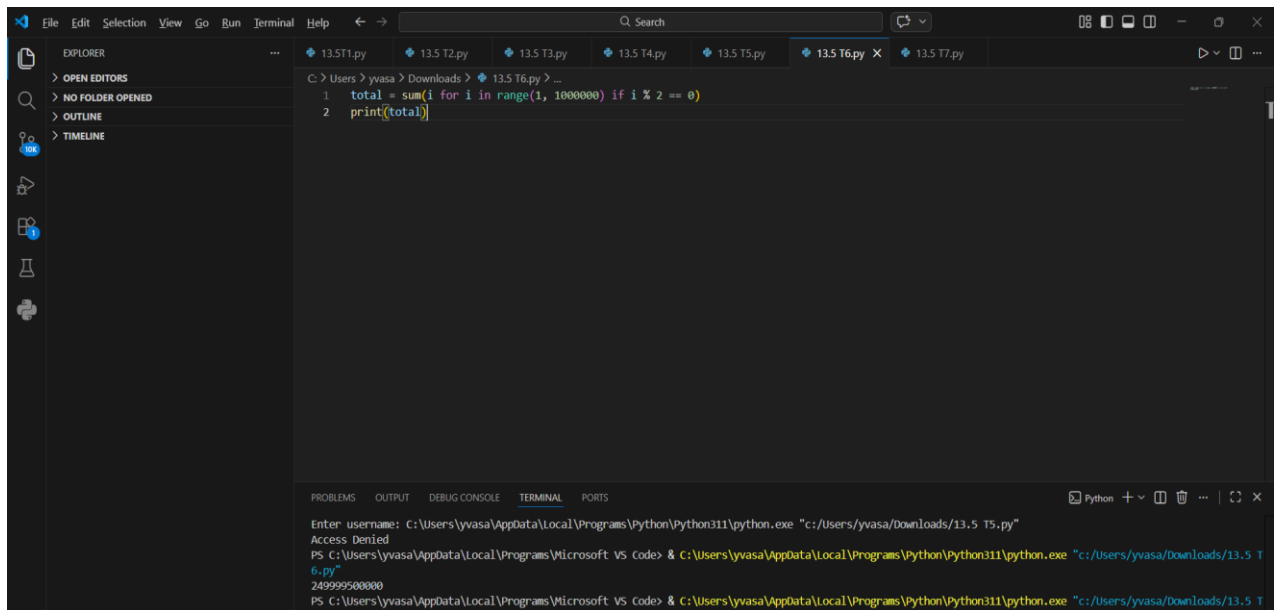
```
    if i % 2 == 0:
```

```
        total += i
```

```
print(total)
```

Expected Outcome:

- o Optimized logic using mathematical formulas or comprehensions, with time comparison.



Task 7 (Removing Hidden Side Effects)

- Task: Refactor code that modifies shared mutable state.

- Focus Areas:

- o Functional-style refactoring

- o Predictability

Legacy Code:

```
data = []
```

```
def add_item(x):
```

```
    data.append(x)
```

```
add_item(10)
```

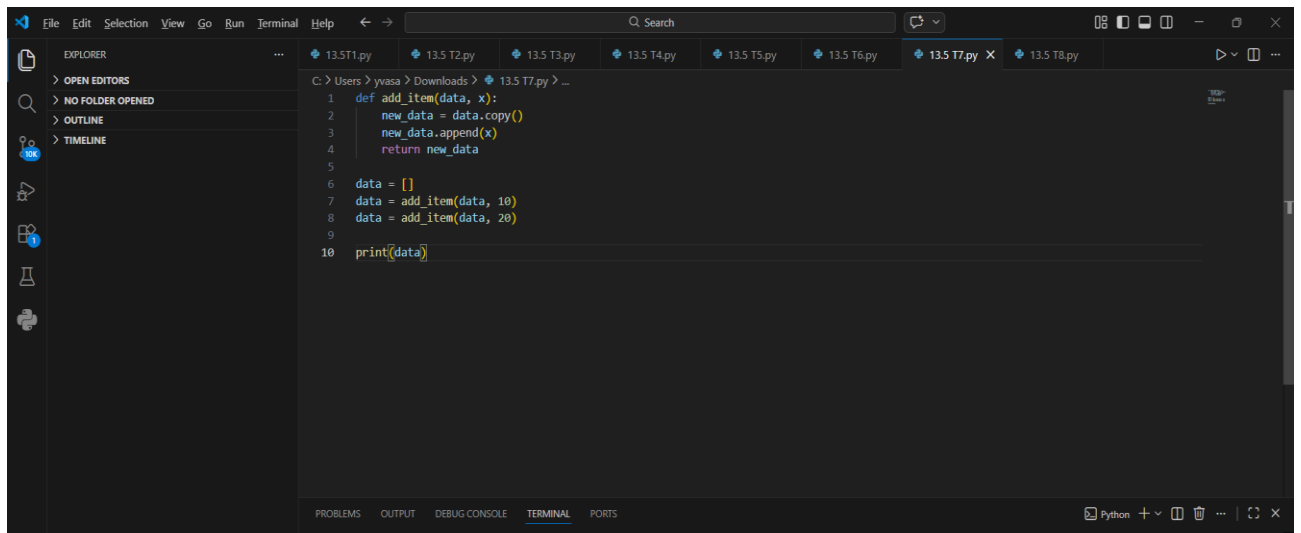
```
add_item(20)
```

```
print(data)
```

Expected Outcome:

- o Refactored function returning values instead of mutating

globals.



```
PS C:\Users\yvasa\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yvasa\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/yvasa/Downloads/13.5 T7.py"
[10, 20]
```

Task 8 (Refactoring Complex Input Validation Logic)

- Task: Use AI to simplify and modularize complex validation rules.

- Focus Areas:

- o Readability

- o Testability

Legacy Code:

```
password = input("Enter password: ")
```

```
if len(password) >= 8:
```

```
if any(c.isdigit() for c in password):
```

```
if any(c.isupper() for c in password):
```

```
print("Valid Password")
```

```
else:
```

```
print("Must contain uppercase")
```

```
else:
```

```
print("Must contain digit")
```

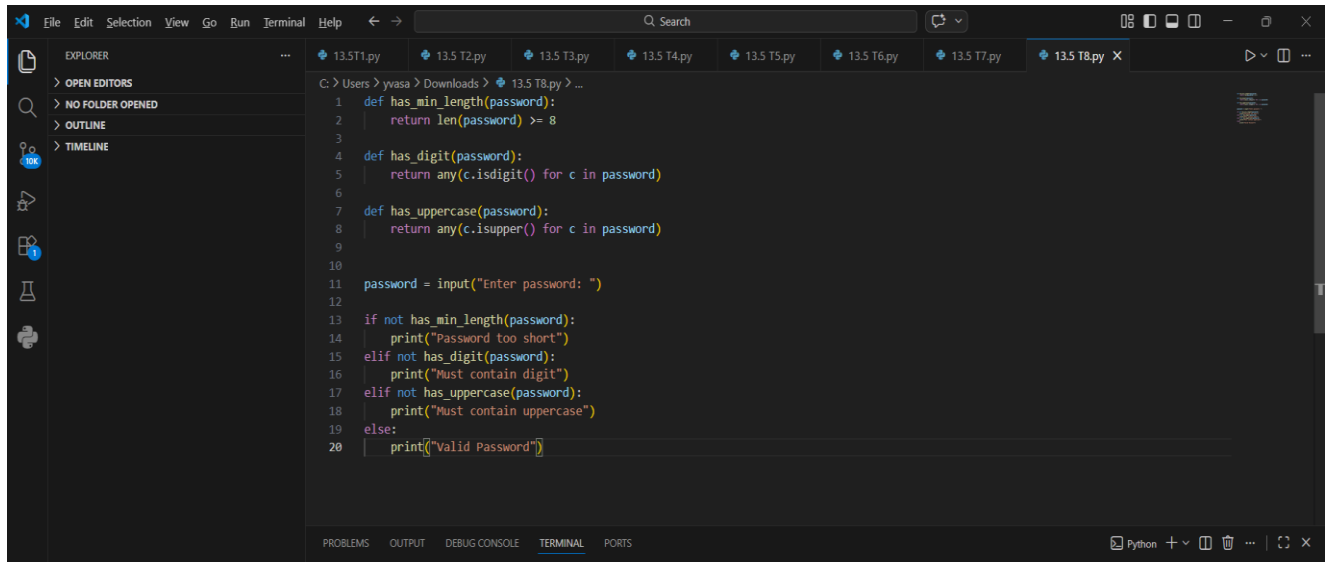
```
else:
```



```
print("Password too short")
```

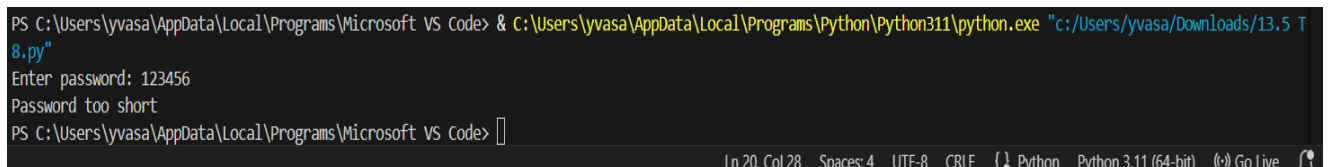
Expected Outcome:

- o Separate validation functions with clear responsibility



The screenshot shows the Visual Studio Code editor interface. The Explorer sidebar on the left shows a project structure with a file named 13.5T8.py. The main editor area displays the code for this file. The code defines three validation functions: has_min_length, has_digit, and has_uppercase. It then prompts the user to enter a password and checks it against these functions. If the password is too short, it prints "Password too short". If it contains a digit, it prints "Must contain digit". If it contains an uppercase letter, it prints "Must contain uppercase". Otherwise, it prints "Valid Password".

```
1 def has_min_length(password):
2     return len(password) >= 8
3
4 def has_digit(password):
5     return any(c.isdigit() for c in password)
6
7 def has_uppercase(password):
8     return any(c.isupper() for c in password)
9
10
11 password = input("Enter password: ")
12
13 if not has_min_length(password):
14     print("Password too short")
15 elif not has_digit(password):
16     print("Must contain digit")
17 elif not has_uppercase(password):
18     print("Must contain uppercase")
19 else:
20     print("Valid Password")
```



The screenshot shows a terminal window with the command prompt. The user has run the command to execute the Python script. The output shows the prompt "Enter password:" followed by the input "123456". The script then prints "Password too short".

```
PS C:\Users\yvasa\AppData\Local\Programs\Microsoft VS Code> & C:\Users\yvasa\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/yvasa/Downloads/13.5 T 8.py"
Enter password: 123456
Password too short
PS C:\Users\yvasa\AppData\Local\Programs\Microsoft VS Code>
```